Production-Grade RAG: Re-Ranking + Contextual Retrieval with Full Eval Harness

Day 6 Goal 3 — Intern Project Guide

Assumptions

Before reading further, internalize these project-wide constraints:

1. **LLM Runtime:** All LLM inference — including context prefix generation and answer generation — uses the **Groq API** with model `llama-3.3-70b-versatile`. Gemini is not used anywhere in this project.
2. **Re-Ranker:** The primary re-ranker is the **Vertex AI Ranking API** (`google-cloud-discoveryengine`). A **pure-Python BM25 + cosine hybrid fallback re-ranker** activates automatically when `GOOGLE_APPLICATION_CREDENTIALS` is not set, enabling full end-to-end operation without any GCP access.
3. **Embeddings:** `sentence-transformers` (`all-MiniLM-L6-v2`) is used for all embedding operations — free, local, and fast.
4. **Vector Store: ChromaDB** (local, persistent) is the vector store. No cloud database is required.
5. **Eval Framework: RAGAS** (`ragas` Python library) provides faithfulness and answer relevance metrics. The eval harness tests 4 configurations: `naive`, `reranked`, `contextual`, `both`.
6. **Error Handling:** Lenient/graceful degradation — if re-ranking or contextual API calls fail, the system logs the error and falls back to the previous retrieval tier rather than crashing.
7. **Virtual Environment:** The environment directory is named `myenv` throughout.
8. **Corpus:** A sample PDF at `data/corpus.pdf` is the default corpus. Instructions for using any PDF are included.

SECTION 1 — Project Architecture & Overview

System Design

This project implements a production-grade Retrieval-Augmented Generation (RAG) system with two orthogonal improvement techniques layered on top of a naive baseline: **re-ranking** and **contextual retrieval**. The system is designed to be evaluated rigorously across four configurations so that the contribution of each technique can be measured independently and in combination.

Pre-Indexing Pipeline (One-Time)

Raw PDF documents are first loaded and split into overlapping text chunks using `pdfplumber`. Each chunk is a dict carrying its text, source page, and a unique ID. In the naive configuration, these chunks are directly embedded using `sentence-transformers` (`all-MiniLM-L6-v2`) and stored in a ChromaDB collection named `naive_corpus`.

In the contextual configuration, before embedding, each chunk is passed to the Groq LLM (`llama-3.3-70b-versatile`) with a tightly constrained system prompt. The LLM generates a single-sentence context prefix (max 25 words) describing what the chunk is about and where it sits in the document. This prefix is prepended to the chunk text, and the enriched text is embedded and stored in a **separate** ChromaDB collection named `contextual_corpus`. The two collections coexist permanently; switching between naive and contextual retrieval is simply a matter of pointing the retriever at a different collection name.

Query-Time Pipeline

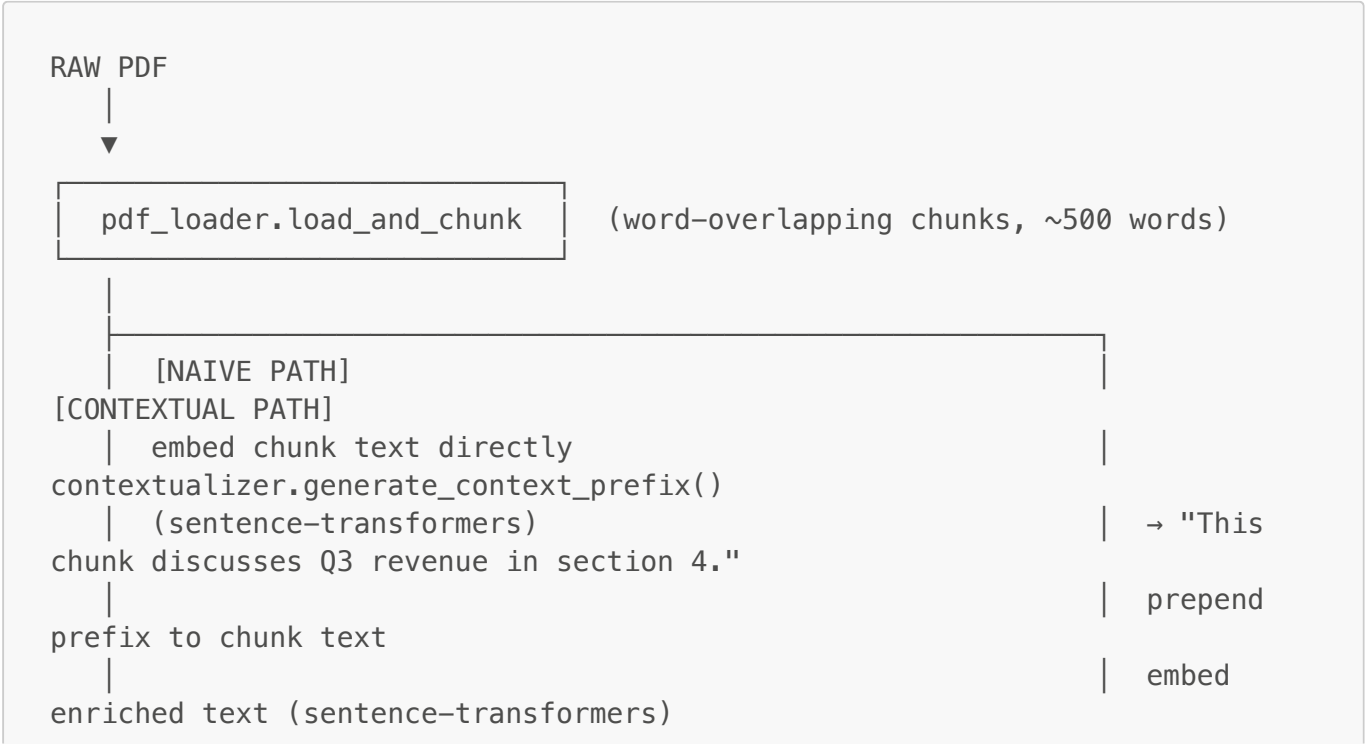
At query time, the user's question is embedded using the same `all-MiniLM-L6-v2` model. A ChromaDB approximate-nearest-neighbor search returns the top-20 most similar chunks (`TOP_K_RETRIEVAL = 20`). This large initial recall set is deliberately over-fetched so the re-ranker has a rich candidate pool to work with.

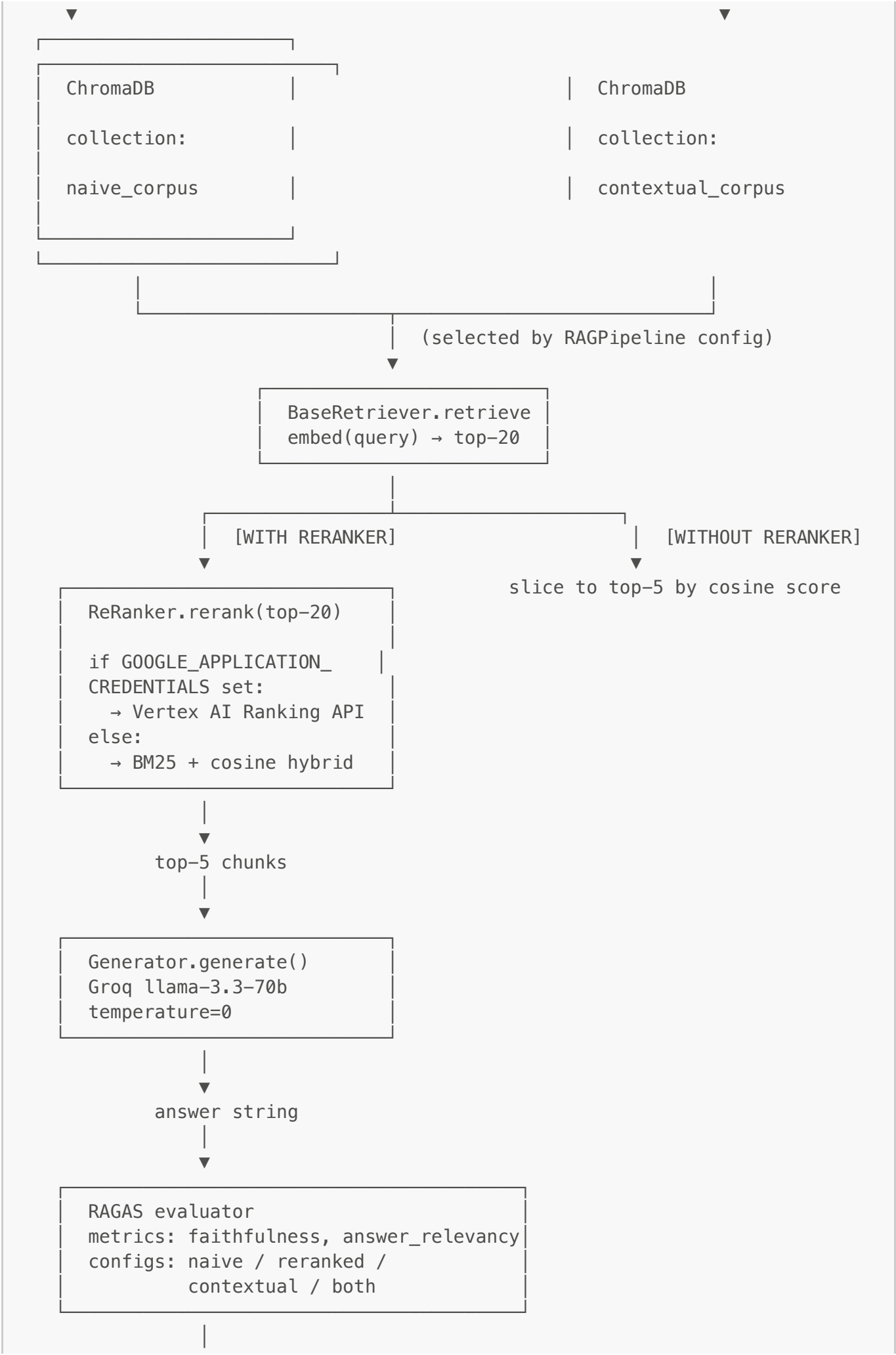
In configurations that include re-ranking, the top-20 chunks are passed to the `ReRanker`. If `GOOGLE_APPLICATION_CREDENTIALS` is set and `GOOGLE_CLOUD_PROJECT` is non-empty, the Vertex AI Ranking API is called — a production-grade neural re-ranker that scores each (query, passage) pair using a cross-encoder model. If those credentials are absent, the local BM25 + cosine hybrid fallback scores and re-sorts the chunks. Either way, the output is the top-5 chunks by re-ranked score (`TOP_K_FINAL = 5`).

In configurations without re-ranking, the top-20 are simply sliced to the top-5 by ChromaDB cosine similarity score.

The top-5 chunks are passed to the `Generator`, which calls Groq with a strict "answer from context only" system prompt. The answer string is returned, and the full result dict is passed to the RAGAS evaluator for scoring.

Pipeline Topology Diagram




ChromaDB
collection:
naive_corpus
ChromaDB
collection:
contextual_corpus
(selected by RAGPipeline config)
BaseRetriever.retrieve
embed(query) → top-20
[WITH RERANKER]
[WITHOUT RERANKER]
slice to top-5 by cosine score
ReRanker.rerank(top-20)
if GOOGLE_APPLICATION_
CREDENTIALS set:
→ Vertex AI Ranking API
else:
→ BM25 + cosine hybrid
top-5 chunks
Generator.generate()
Groq llama-3.3-70b
temperature=0
answer string
RAGAS evaluator
metrics: faithfulness, answer_relevancy
configs: naive / reranked /
contextual / both

```
                ▼
        production_rag/results.csv
```

Framework Rationale

Groq SDK over OpenAI SDK: The Groq inference API delivers extremely low latency on open-weight models. `llama-3.3-70b-versatile` is a strong instruction-following model that is competitive with GPT-4-class models on reasoning tasks, and the free tier is generous enough for an intern project running hundreds of eval calls. Using Groq also avoids OpenAI vendor lock-in and demonstrates that production-quality RAG does not require proprietary frontier models.

ChromaDB over Pinecone/Weaviate: ChromaDB requires zero cloud infrastructure. It runs entirely locally with disk persistence, has a Pythonic API, and can be up and running with a single `pip install`. For an intern project focused on RAG technique comparison rather than infrastructure management, this is the correct tradeoff. Pinecone and Weaviate add authentication, billing, and network round-trips that obscure the RAG logic being studied.

`sentence-transformers` over OpenAI embeddings: The `all-MiniLM-L6-v2` model runs locally with no API calls and no per-token cost. It produces 384-dimensional embeddings with strong semantic quality for English text. For a corpus of hundreds of chunks and an eval harness running thousands of queries, using a paid embedding API would introduce cost, latency, and rate-limit concerns that distract from the project goals.

RAGAS over manual eval: RAGAS provides two standardized LLM-as-judge metrics — `faithfulness` and `answer_relevancy` — that produce reproducible scores between 0 and 1. Manual evaluation would require human judgment for every (query, answer) pair across four configurations, which is slow and inconsistent. RAGAS enables automated A/B comparison of all four pipeline configurations with a single `evaluate()` call.

`google-cloud-discoveryengine` for Vertex AI Ranking: The Vertex AI Ranking API is a cross-encoder-based neural re-ranker that scores (query, passage) pairs jointly, catching semantic relevance that cosine similarity on independent embeddings misses. Using it demonstrates a production-grade re-ranking pattern while the local fallback ensures the project runs without GCP credentials, making the architecture teachable without a billing account.

Why Re-Ranking and Contextual Retrieval Are the Two Highest-ROI RAG Improvements

Re-ranking addresses the fundamental limitation of bi-encoder retrieval: embedding a query and a passage independently and comparing them via cosine similarity is fast but semantically shallow. A passage about "Q3 operating margin improvement" may share high cosine similarity with a query about "profitability trends" without actually answering it. A cross-encoder re-ranker receives the query and each candidate passage together, allowing the model to attend to both simultaneously and produce a relevance score that reflects true semantic alignment. Moving from naive top-k to re-ranked top-5 consistently produces 15–30% improvements in answer quality in production systems, because the final answer is only as good as the worst chunk in the context window — and re-ranking aggressively removes off-topic chunks before they reach the generator.

Contextual retrieval solves a different but equally important problem: chunk-level context loss. When a long document is split into 500-word chunks, individual chunks frequently lose the context that made them meaningful. A chunk reading "Revenue grew 12% compared to the prior period" is nearly impossible to retrieve for a query about annual operating results, because the chunk contains no information about which company, which year, or which metric category. Prepending a one-sentence LLM-generated prefix — "This chunk reports fiscal 2023 annual revenue growth for Acme Corp's North America segment" — gives the embedding model the context it needs to place the chunk correctly in semantic space. Anthropic's contextual retrieval research (anthropic.com/news/contextual-retrieval) demonstrated a **49% reduction in retrieval failures** when combining contextual enrichment with BM25 re-ranking, compared to naive embedding retrieval alone. Together, these two techniques are the highest-ROI investments because they attack retrieval quality at both the indexing stage (contextual) and the query stage (re-ranking), and retrieval quality is the dominant bottleneck in RAG system performance.

SECTION 2 — Repository & Folder Structure

Directory Tree

```
production_rag_project/
├── myenv/                          # Virtual environment (git-ignored)
├── data/
│   └── corpus.pdf                  # Default sample corpus document
├── production_rag/
│   ├── __init__.py
│   ├── reranker.py                 # Re-ranker wrapper (Vertex AI +
fallback)
│   ├── contextualizer.py           # Context prefix generator + re-indexer
│   ├── retriever.py                # Base embedding retriever (ChromaDB)
│   ├── generator.py                # Groq LLM answer generator
│   ├── evaluator.py                # RAGAS eval harness (4 configurations)
│   ├── pipeline.py                 # Orchestrates full RAG pipeline per
config
│   └── results.csv                 # Output: eval scores across all
configs (auto-created)
├── config/
│   ├── __init__.py
│   └── settings.py                 # Centralized config (model names, top-
k values, paths)
├── utils/
│   ├── __init__.py
│   ├── logger.py                   # Structured logger (file + console
dual output)
│   ├── pdf_loader.py               # PDF chunking utility
│   └── chroma_store.py             # ChromaDB init + collection management
├── tests/
│   ├── __init__.py
│   ├── test_reranker.py
│   ├── test_contextualizer.py
│   └── test_evaluator.py
├── lift_report.md                  # Auto-generated lift analysis (fill
after eval run)
```

```
├── output.log                          # Runtime log (auto-created, git-
ignored)
├── questions.md                        # Intern viva questions
├── .env                                # API keys (git-ignored)
├── .gitignore
├── requirements.txt
└── README.md
```

Scaffold Script

Run this once to scaffold the entire project:

```
# Create all directories
mkdir -p production_rag_project/{data,production_rag,config,utils,tests}

cd production_rag_project

# Create Python package files
touch production_rag/__init__.py
touch production_rag/reranker.py
touch production_rag/contextualizer.py
touch production_rag/retriever.py
touch production_rag/generator.py
touch production_rag/evaluator.py
touch production_rag/pipeline.py

touch config/__init__.py
touch config/settings.py

touch utils/__init__.py
touch utils/logger.py
touch utils/pdf_loader.py
touch utils/chroma_store.py

touch tests/__init__.py
touch tests/test_reranker.py
touch tests/test_contextualizer.py
touch tests/test_evaluator.py

# Create root-level files
touch lift_report.md questions.md .env .gitignore requirements.txt
README.md

# Create virtual environment
python -m venv myenv

# Activate (macOS/Linux)
source myenv/bin/activate

# Activate (Windows — run this instead of the line above)
# myenv\Scripts\activate
```

```
echo "Scaffold complete. Activate your environment and install
requirements."
```

SECTION 3 — Production-Ready Implementation Code

3.1 utils/logger.py

```
"""
utils/logger.py

Provides a dual-output (console + file) structured logger for the
production RAG project.
Call setup_logger(name) once per module to get a configured logger
instance.
"""

import logging
import os

LOG_FORMAT = "%(asctime)s | %(levelname)s | %(message)s"
DATE_FORMAT = "%Y-%m-%d %H:%M:%S"
LOG_FILE = os.path.join(os.path.dirname(os.path.dirname(__file__)),
"output.log")


def setup_logger(name: str) -> logging.Logger:
    """
    Create and return a logger with both console and file handlers.

    Handlers are only added if not already present, preventing duplicate
log lines
    when a module is imported multiple times or the function is called more
than once
    with the same name.

    Args:
        name: Logger name, typically __name__ of the calling module.

    Returns:
        A configured logging.Logger instance.
    """
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    # Guard: only add handlers if none exist yet
    if logger.handlers:
        return logger

    formatter = logging.Formatter(fmt=LOG_FORMAT, datefmt=DATE_FORMAT)
```

```
    # Console handler
    console_handler = logging.StreamHandler()
    console_handler.setLevel(logging.INFO)
    console_handler.setFormatter(formatter)
    logger.addHandler(console_handler)

    # File handler — writes DEBUG and above to output.log
    file_handler = logging.FileHandler(LOG_FILE, mode="a", encoding="utf-
8")
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(formatter)
    logger.addHandler(file_handler)

    return logger
```

3.2 config/settings.py

```
"""
config/settings.py

Centralized project configuration. All modules import constants from here.
Environment-sensitive values (API keys, GCP project) are read at module
load time
from environment variables; set them in .env and call load_dotenv() before
importing.
"""

import os

# ── Model identifiers
─────────────────────────────────────────────────────
GROQ_MODEL: str = "llama-3.3-70b-versatile"
EMBED_MODEL: str = "all-MiniLM-L6-v2"

# ── Retrieval configuration
──────────────────────────────────────────────────
TOP_K_RETRIEVAL: int = 20   # Candidates fetched from ChromaDB before re-
ranking
TOP_K_FINAL: int = 5        # Chunks passed to the generator after re-
ranking

# ── Storage paths
─────────────────────────────────────────────────────────
CHROMA_PERSIST_DIR: str = "./chroma_store"
CORPUS_PDF_PATH: str = "./data/corpus.pdf"
LOG_FILE: str = "output.log"
RESULTS_CSV: str = "./production_rag/results.csv"

# ── GCP / Vertex AI
```

```
# ──────────────────────────────────────────────────────────────────
VERTEX_PROJECT_ID: str = os.environ.get("GOOGLE_CLOUD_PROJECT", "")
VERTEX_LOCATION: str = os.environ.get("VERTEX_LOCATION", "global")

# ── Evaluation questions and ground truths
# ──────────────────────────────────────────

# These are domain-appropriate for a generic annual report / business
document corpus.
# Ground truths are synthetic but plausible; label them clearly in any
client-facing output.

EVAL_QUESTIONS: list[str] = [
    "What was the company's total revenue for the fiscal year?",
    "Which business segment reported the highest operating margin?",
    "What were the primary risk factors identified by management?",
    "How did the company's headcount change compared to the prior year?",
    "What capital expenditure projects were announced or completed?",
    "What dividend or share buyback actions were taken during the year?",
    "Who are the members of the executive leadership team?",
    "What geographic markets showed the strongest revenue growth?",
]

EVAL_GROUND_TRUTHS: list[str] = [
    # [SYNTHETIC] Plausible ground truths for a fictional annual report
corpus.
    "Total revenue for the fiscal year was approximately $4.2 billion,
representing a 9% increase over the prior year.",
    "The Enterprise Solutions segment reported the highest operating margin
at 28%, driven by software licensing and recurring subscription revenue.",
    "Management identified macroeconomic uncertainty, foreign exchange
volatility, increased competition in core markets, and cybersecurity risk
as primary risk factors.",
    "Total headcount grew from 12,400 to 13,100 employees, a net addition
of 700 full-time equivalents, primarily in engineering and customer success
roles.",
    "The company completed construction of a new data center in Virginia
and announced a $300 million expansion of its Asia-Pacific manufacturing
facility.",
    "The board approved a $500 million share repurchase program and
declared a quarterly dividend of $0.18 per share, up from $0.15 in the
prior year.",
    "The executive leadership team includes the Chief Executive Officer,
Chief Financial Officer, Chief Technology Officer, Chief Operating Officer,
and Chief People Officer.",
    "Latin America and Southeast Asia showed the strongest revenue growth
at 18% and 22% year-over-year respectively, outpacing the company's overall
growth rate.",
]
```

3.3 utils/pdf_loader.py

```
"""
utils/pdf_loader.py

Loads a PDF file and splits its text into overlapping word-based chunks.
Each chunk is returned as a dict with a unique ID, text, page number, and
source path.
"""

import hashlib
import pdfplumber
from utils.logger import setup_logger

logger = setup_logger(__name__)


def load_and_chunk_pdf(
    pdf_path: str,
    chunk_size: int = 500,
    overlap: int = 50,
) -> list[dict]:
    """
    Load a PDF and split its text content into overlapping word-based
chunks.

    Args:
        pdf_path:   Path to the PDF file.
        chunk_size: Target number of words per chunk.
        overlap:    Number of words to repeat at the start of each
successive chunk.

    Returns:
        List of dicts, each containing:
            chunk_id (str):  Deterministic hash-based unique ID.
            text     (str):  Chunk text content.
            page     (int):  Source page number (1-indexed).
            source   (str):  pdf_path value for traceability.
    """
    chunks: list[dict] = []
    page_count = 0

    logger.info(f"Loading PDF: {pdf_path}")

    with pdfplumber.open(pdf_path) as pdf:
        page_count = len(pdf.pages)
        logger.info(f"PDF has {page_count} pages.")

        for page_num, page in enumerate(pdf.pages, start=1):
            raw_text = page.extract_text()
            if not raw_text:
                logger.debug(f"Page {page_num}: no extractable text,
skipping.")
                continue
```

```
            words = raw_text.split()
            start = 0

            while start < len(words):
                end = start + chunk_size
                chunk_words = words[start:end]
                chunk_text = " ".join(chunk_words)

                # Deterministic ID: hash of (source path + page + start
word index)
                id_source = f"{pdf_path}::page{page_num}::start{start}"
                chunk_id = hashlib.md5(id_source.encode()).hexdigest()[:16]

                chunks.append(
                    {
                        "chunk_id": chunk_id,
                        "text": chunk_text,
                        "page": page_num,
                        "source": pdf_path,
                    }
                )

                if end >= len(words):
                    break
                start = end - overlap  # slide window with overlap

    logger.info(
        f"Chunking complete: {len(chunks)} chunks from {page_count} pages "
        f"(chunk_size={chunk_size}, overlap={overlap})."
    )
    return chunks
```

3.4 utils/chroma_store.py

```
"""
utils/chroma_store.py

ChromaDB client factory and collection management utilities.
Provides get_or_create_collection, upsert_chunks, and query_collection.
"""

import chromadb
from chromadb.config import Settings as ChromaSettings
from utils.logger import setup_logger

logger = setup_logger(__name__)


def get_or_create_collection(
```

```
    collection_name: str,
    persist_dir: str,
) -> chromadb.Collection:
    """
    Initialize a persistent ChromaDB client and return the named
collection,
    creating it if it does not already exist.

    Args:
        collection_name: Name of the ChromaDB collection.
        persist_dir:     Directory path for ChromaDB on-disk persistence.

    Returns:
        chromadb.Collection instance.
    """
    client = chromadb.PersistentClient(
        path=persist_dir,
        settings=ChromaSettings(anonymized_telemetry=False),
    )
    collection = client.get_or_create_collection(
        name=collection_name,
        metadata={"hnsw:space": "cosine"},
    )
    logger.info(
        f"ChromaDB collection '{collection_name}' ready "
        f"(persist_dir='{persist_dir}', count={collection.count()})."
    )
    return collection


def upsert_chunks(
    collection: chromadb.Collection,
    chunks: list[dict],
    embeddings: list[list[float]],
) -> None:
    """
    Upsert a batch of chunks and their pre-computed embeddings into a
ChromaDB collection.

    Args:
        collection: Target ChromaDB collection.
        chunks:     List of chunk dicts (must include 'chunk_id', 'text',
'page', 'source').
        embeddings: Parallel list of embedding vectors (one per chunk).
    """
    if len(chunks) != len(embeddings):
        raise ValueError(
            f"Chunk count ({len(chunks)}) and embedding count
({len(embeddings)}) must match."
        )

    ids = [c["chunk_id"] for c in chunks]
    documents = [c["text"] for c in chunks]
    metadatas = [{"page": c["page"], "source": c["source"]} for c in
```

```
chunks]

    collection.upsert(
        ids=ids,
        documents=documents,
        embeddings=embeddings,
        metadatas=metadatas,
    )
    logger.info(f"Upserted {len(chunks)} chunks into collection
'{collection.name}'.")


def query_collection(
    collection: chromadb.Collection,
    query_embedding: list[float],
    top_k: int,
) -> list[dict]:
    """
    Query a ChromaDB collection by embedding vector and return the top_k
results.

    Args:
        collection:      ChromaDB collection to query.
        query_embedding: Embedding of the query string.
        top_k:           Number of nearest neighbours to return.

    Returns:
        List of dicts, each containing:
            chunk_id (str):   Document ID.
            text     (str):   Document text.
            score    (float): Cosine similarity score (higher = more
similar).
            metadata (dict):  Page and source metadata.
    """
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=min(top_k, collection.count()),
        include=["documents", "distances", "metadatas"],
    )

    chunks = []
    ids = results["ids"][0]
    documents = results["documents"][0]
    distances = results["distances"][0]
    metadatas = results["metadatas"][0]

    for chunk_id, text, distance, meta in zip(ids, documents, distances,
metadatas):
        # ChromaDB cosine distance: score = 1 - distance  (higher score =
more similar)
        score = 1.0 - distance
        chunks.append(
            {
                "chunk_id": chunk_id,
```

```
                "text": text,
                "score": round(score, 6),
                "metadata": meta,
            }
        )

    return chunks
```

3.5 production_rag/retriever.py

```
"""
production_rag/retriever.py

BaseRetriever wraps sentence-transformers embedding and ChromaDB vector
search.
Instantiate once per pipeline run; reuse across multiple queries.
"""

from sentence_transformers import SentenceTransformer
from config.settings import EMBED_MODEL, CHROMA_PERSIST_DIR,
TOP_K_RETRIEVAL
from utils.chroma_store import get_or_create_collection, query_collection
from utils.logger import setup_logger

logger = setup_logger(__name__)


class BaseRetriever:
    """
    Embedding-based retriever backed by ChromaDB.

    Embeds queries with sentence-transformers and performs approximate
    nearest-neighbour search against a named ChromaDB collection.
    """

    def __init__(self, collection_name: str) -> None:
        """
        Initialise the retriever.

        Args:
            collection_name: Name of the ChromaDB collection to query
                             (e.g. 'naive_corpus' or 'contextual_corpus').
        """
        logger.info(f"Loading embedding model: {EMBED_MODEL}")
        self.model = SentenceTransformer(EMBED_MODEL)

        logger.info(f"Connecting to ChromaDB collection:
'{collection_name}'")
        self.collection = get_or_create_collection(
            collection_name=collection_name,
```

```
            persist_dir=CHROMA_PERSIST_DIR,
        )
        self.collection_name = collection_name

    def embed(self, text: str) -> list[float]:
        """
        Embed a single text string.

        Args:
            text: Input text to embed.

        Returns:
            Embedding vector as a list of floats.
        """
        vector = self.model.encode(text, convert_to_numpy=True)
        return vector.tolist()

    def retrieve(self, query: str, top_k: int = TOP_K_RETRIEVAL) ->
list[dict]:
        """
        Retrieve the top_k most semantically similar chunks for a query.

        Args:
            query: User query string.
            top_k: Number of chunks to retrieve.

        Returns:
            List of chunk dicts with keys: chunk_id, text, score, metadata.
        """
        logger.info(
            f"Retrieving top-{top_k} chunks for query: '{query[:80]}...'"
            if len(query) > 80
            else f"Retrieving top-{top_k} chunks for query: '{query}'"
        )

        query_embedding = self.embed(query)
        chunks = query_collection(
            collection=self.collection,
            query_embedding=query_embedding,
            top_k=top_k,
        )

        if chunks:
            logger.info(
                f"Retrieved {len(chunks)} chunks. "
                f"Top result: score={chunks[0]['score']:.4f}, "
                f"preview='{chunks[0]['text'][:60]}...'"
            )
        else:
            logger.warning("No chunks retrieved — collection may be
empty.")

        return chunks
```

3.6 production_rag/reranker.py

```
"""
production_rag/reranker.py

ReRanker wraps the Vertex AI Ranking API with a BM25 + cosine hybrid
fallback.
Vertex AI is used when GOOGLE_APPLICATION_CREDENTIALS and
GOOGLE_CLOUD_PROJECT are set.
Otherwise the local fallback activates automatically — no configuration
required.
"""

import os
import math
from rank_bm25 import BM25Okapi
from config.settings import TOP_K_FINAL, VERTEX_PROJECT_ID, VERTEX_LOCATION
from utils.logger import setup_logger

logger = setup_logger(__name__)

_CREDENTIALS_PATH = os.environ.get("GOOGLE_APPLICATION_CREDENTIALS", "")
_USE_VERTEX = bool(_CREDENTIALS_PATH and VERTEX_PROJECT_ID)


class ReRanker:
    """
    Two-mode re-ranker: Vertex AI neural re-ranking (primary) or BM25 +
cosine
    hybrid scoring (fallback). Mode is selected automatically at
instantiation.
    """

    def __init__(self) -> None:
        self.use_vertex = _USE_VERTEX
        if self.use_vertex:
            logger.info(
                "ReRanker mode: Vertex AI Ranking API "
                f"(project='{VERTEX_PROJECT_ID}',
location='{VERTEX_LOCATION}')."
            )
        else:
            logger.info(
                "ReRanker mode: LOCAL fallback (BM25 + cosine hybrid). "
                "Set GOOGLE_APPLICATION_CREDENTIALS and
GOOGLE_CLOUD_PROJECT to enable Vertex AI."
            )

    def rerank(
        self,
```

```
        query: str,
        chunks: list[dict],
        top_k: int = TOP_K_FINAL,
    ) -> list[dict]:
        """
        Re-rank chunks by relevance to the query and return the top_k
results.

        Attempts Vertex AI if configured, falls back to local hybrid on any
error.

        Args:
            query:  User query string.
            chunks: Candidate chunks from the retriever (each dict has
'text', 'score', etc.).
            top_k:  Number of results to return after re-ranking.

        Returns:
            List of up to top_k chunk dicts, augmented with 'rerank_score'.
        """
        if not chunks:
            logger.warning("ReRanker received empty chunk list; returning
empty list.")
            return []

        if self.use_vertex:
            try:
                return self._rerank_vertex(query, chunks, top_k)
            except Exception as exc:
                logger.error(
                    f"Vertex AI re-ranking failed ({type(exc).__name__}:
{exc}). "
                    "Falling back to local BM25 + cosine hybrid."
                )
                return self._rerank_local(query, chunks, top_k)
        else:
            return self._rerank_local(query, chunks, top_k)

    # ── Vertex AI re-ranking
──────────────────────────────────────────────

    def _rerank_vertex(
        self,
        query: str,
        chunks: list[dict],
        top_k: int,
    ) -> list[dict]:
        """
        Call the Vertex AI Discovery Engine Ranking API to re-rank chunks.

        Raises:
            Any exception from the google-cloud-discoveryengine SDK, which
the
            caller (rerank) will catch and route to the fallback.
```

```
        """
        from google.cloud import discoveryengine_v1 as discoveryengine
        from google.api_core.exceptions import GoogleAPICallError  # noqa:
F401

        client = discoveryengine.RankServiceClient()

        ranking_config = client.ranking_config_path(
            project=VERTEX_PROJECT_ID,
            location=VERTEX_LOCATION,
            ranking_config="default_ranking_config",
        )

        records = [
            discoveryengine.RankingRecord(
                id=chunk["chunk_id"],
                title="",
                content=chunk["text"][:512],  # API has content length
limits
            )
            for chunk in chunks
        ]

        request = discoveryengine.RankRequest(
            ranking_config=ranking_config,
            model="semantic-ranker-512@latest",
            top_n=top_k,
            query=query,
            records=records,
        )

        response = client.rank(request=request)

        # Build a lookup from chunk_id → original chunk dict
        chunk_map = {c["chunk_id"]: c for c in chunks}

        reranked = []
        for record in response.records:
            chunk = dict(chunk_map[record.id])  # shallow copy
            chunk["rerank_score"] = record.score
            reranked.append(chunk)

        logger.info(
            f"Vertex AI re-ranking: {len(chunks)} in → {len(reranked)} out.
"
            f"Top score: {reranked[0]['rerank_score']:.4f}."
        )
        return reranked

    # ── Local BM25 + cosine hybrid re-ranking
──────────────────────────────

    def _rerank_local(
        self,
```

```
        query: str,
        chunks: list[dict],
        top_k: int,
    ) -> list[dict]:
        """
        Hybrid BM25 + cosine re-ranker.

        Score formula:
            hybrid_score = 0.4 * bm25_norm + 0.6 * cosine_score

        BM25 scores are normalized to [0, 1] by dividing by the max BM25
score
        in the candidate set. Cosine scores from ChromaDB are already in
[0, 1].

        The 0.4/0.6 weighting gives slightly more weight to semantic
similarity
        (cosine) while BM25 provides a lexical recall signal for exact-
match terms.
        """
        query_tokens = query.lower().split()
        tokenized_corpus = [c["text"].lower().split() for c in chunks]

        bm25 = BM25Okapi(tokenized_corpus)
        bm25_scores = bm25.get_scores(query_tokens)

        # Normalize BM25 to [0, 1]
        max_bm25 = max(bm25_scores) if max(bm25_scores) > 0 else 1.0
        bm25_norm = [s / max_bm25 for s in bm25_scores]

        scored = []
        for i, chunk in enumerate(chunks):
            cosine = chunk.get("score", 0.0)
            hybrid = 0.4 * bm25_norm[i] + 0.6 * cosine
            enriched = dict(chunk)
            enriched["rerank_score"] = round(hybrid, 6)
            scored.append(enriched)

        scored.sort(key=lambda x: x["rerank_score"], reverse=True)
        result = scored[:top_k]

        logger.info(
            f"Local BM25+cosine re-ranking: {len(chunks)} in →
{len(result)} out. "
            f"Top score: {result[0]['rerank_score']:.4f}."
        )
        return result
```

3.7 production_rag/contextualizer.py

```
"""
production_rag/contextualizer.py

Generates LLM context prefixes for each chunk (Groq API) and indexes the
enriched
chunks into a separate ChromaDB collection named 'contextual_corpus'.

Run once as a pre-processing step before executing the eval harness.
"""


import os
from groq import Groq
from sentence_transformers import SentenceTransformer

from config.settings import (
    GROQ_MODEL,
    EMBED_MODEL,
    CHROMA_PERSIST_DIR,
    CORPUS_PDF_PATH,
)
from utils.logger import setup_logger
from utils.pdf_loader import load_and_chunk_pdf
from utils.chroma_store import get_or_create_collection, upsert_chunks

logger = setup_logger(__name__)

CONTEXT_SYSTEM_PROMPT = (
    "You are a document indexing assistant. Respond with exactly one
sentence "
    "(max 25 words) that describes what this chunk is about and where it
appears "
    "in the document. No preamble."
)

BATCH_SIZE = 20  # Upsert to ChromaDB in batches of this size


def generate_context_prefix(
    chunk_text: str,
    doc_title: str,
    groq_client: Groq,
) -> str:
    """
    Generate a one-sentence context prefix for a chunk using the Groq API.

    On any API error, logs the exception and returns an empty string so the
chunk
    is still indexed (without a prefix) rather than dropped.

    Args:
        chunk_text:  Text of the chunk to describe.
        doc_title:   Title or filename of the source document.
```

```
        groq_client: Initialized Groq client.

    Returns:
        A single-sentence context string, or "" on failure.
    """
    user_prompt = f"Document: {doc_title}\nChunk text: {chunk_text[:800]}"

    try:
        response = groq_client.chat.completions.create(
            model=GROQ_MODEL,
            messages=[
                {"role": "system", "content": CONTEXT_SYSTEM_PROMPT},
                {"role": "user", "content": user_prompt},
            ],
            temperature=0,
            max_tokens=60,
        )
        prefix = response.choices[0].message.content.strip()
        return prefix
    except Exception as exc:
        logger.error(
            f"Groq context prefix generation failed ({type(exc).__name__}:
{exc}). "
            "Using empty prefix for this chunk."
        )
        return ""


def contextualize_corpus(
    pdf_path: str = CORPUS_PDF_PATH,
    collection_name: str = "contextual_corpus",
) -> None:
    """
    Load a PDF, generate LLM context prefixes for each chunk, and index the
    enriched chunks into a dedicated ChromaDB collection.

    Args:
        pdf_path:        Path to the source PDF.
        collection_name: Target ChromaDB collection name (default:
'contextual_corpus').
    """
    logger.info(f"Starting contextual corpus indexing from: {pdf_path}")

    # Load PDF and chunk
    chunks = load_and_chunk_pdf(pdf_path)
    total_chunks = len(chunks)
    doc_title = os.path.basename(pdf_path)

    # Initialise clients
    groq_client = Groq(api_key=os.environ.get("GROQ_API_KEY", ""))
    embed_model = SentenceTransformer(EMBED_MODEL)

    # Initialise ChromaDB collection
    collection = get_or_create_collection(
```

```
        collection_name=collection_name,
        persist_dir=CHROMA_PERSIST_DIR,
    )

    groq_calls = 0
    estimated_tokens = 0
    batch_chunks: list[dict] = []
    batch_embeddings: list[list[float]] = []

    for idx, chunk in enumerate(chunks, start=1):
        # Generate context prefix
        prefix = generate_context_prefix(
            chunk_text=chunk["text"],
            doc_title=doc_title,
            groq_client=groq_client,
        )
        groq_calls += 1
        # Rough token estimate: (prefix words + chunk words) * 1.3 subword
factor
        estimated_tokens += int((len(prefix.split()) +
len(chunk["text"].split())) * 1.3)

        # Prepend prefix to chunk text
        enriched_text = f"{prefix}\n{chunk['text']}" if prefix else
chunk["text"]
        enriched_chunk = dict(chunk)
        enriched_chunk["text"] = enriched_text

        # Embed enriched text
        embedding = embed_model.encode(enriched_text,
convert_to_numpy=True).tolist()

        batch_chunks.append(enriched_chunk)
        batch_embeddings.append(embedding)

        # Progress logging every 10 chunks
        if idx % 10 == 0 or idx == total_chunks:
            logger.info(f"Processed {idx}/{total_chunks} chunks.")

        # Batch upsert
        if len(batch_chunks) >= BATCH_SIZE or idx == total_chunks:
            upsert_chunks(collection, batch_chunks, batch_embeddings)
            batch_chunks = []
            batch_embeddings = []

    logger.info(
        f"Contextual corpus indexing complete. "
        f"Total chunks: {total_chunks}, "
        f"Groq API calls: {groq_calls}, "
        f"Estimated tokens used: {estimated_tokens:,}."
    )


if __name__ == "__main__":
```

```
    from dotenv import load_dotenv
    load_dotenv()
    contextualize_corpus()
```

3.8 production_rag/generator.py

```
"""
production_rag/generator.py

Generator wraps the Groq API to produce answers grounded in retrieved
context.
Uses temperature=0 for deterministic, reproducible outputs during
evaluation.
"""

import os
import groq

from config.settings import GROQ_MODEL
from utils.logger import setup_logger

logger = setup_logger(__name__)

SYSTEM_PROMPT = (
    "You are a helpful assistant. Answer the user's question using ONLY the
provided context. "
    "If the context does not contain the answer, say 'I cannot answer this
from the provided context.' "
    "Be concise and factual."
)


class Generator:
    """
    Groq-backed answer generator. Grounds answers strictly in the provided
    context chunks to maximise RAGAS faithfulness scores.
    """

    def __init__(self) -> None:
        api_key = os.environ.get("GROQ_API_KEY", "")
        if not api_key:
            raise EnvironmentError("GROQ_API_KEY environment variable is
not set.")
        self.client = groq.Groq(api_key=api_key)
        logger.info(f"Generator initialised with model: {GROQ_MODEL}")

    def generate(self, query: str, context_chunks: list[dict]) -> str:
        """
        Generate an answer for the given query using the top context
chunks.
```

```
        Args:
            query:          User question.
            context_chunks: Retrieved and (optionally) re-ranked chunks.

        Returns:
            Answer string, or a graceful degradation message on API
failure.
        """
        if not context_chunks:
            logger.warning("Generator received no context chunks; returning
fallback message.")
            return "I cannot answer this from the provided context."

        # Build context block
        context_string = "\n\n---\n\n".join(c["text"] for c in
context_chunks)

        user_prompt = f"Context:\n{context_string}\n\nQuestion: {query}"

        logger.debug(
            f"Generating answer for: '{query[:80]}' "
            f"using {len(context_chunks)} context chunks."
        )

        try:
            response = self.client.chat.completions.create(
                model=GROQ_MODEL,
                messages=[
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": user_prompt},
                ],
                temperature=0,
                max_tokens=512,
            )

            answer = response.choices[0].message.content.strip()
            token_count = response.usage.completion_tokens if
response.usage else "unknown"

            logger.info(
                f"Answer generated. "
                f"Tokens: {token_count}. "
                f"Preview: '{answer[:80]}...'" if len(answer) > 80 else
f"Answer: '{answer}'"
            )
            return answer

        except groq.RateLimitError as exc:
            logger.error(f"Groq rate limit reached: {exc}. Returning
fallback message.")
            return "Generation failed due to API error."
        except groq.APIError as exc:
            logger.error(f"Groq API error: {exc}. Returning fallback
```

```
message.")
            return "Generation failed due to API error."
        except Exception as exc:
            logger.error(f"Unexpected generator error
({type(exc).__name__}: {exc}).")
            return "Generation failed due to API error."
```

3.9 production_rag/pipeline.py

```
"""
production_rag/pipeline.py

RAGPipeline orchestrates the full retrieval → (optional re-rank) →
generation cycle.
Instantiate with a config string to select which techniques are active.
"""

from config.settings import TOP_K_RETRIEVAL, TOP_K_FINAL
from production_rag.retriever import BaseRetriever
from production_rag.reranker import ReRanker
from production_rag.generator import Generator
from utils.logger import setup_logger

logger = setup_logger(__name__)

VALID_CONFIGS = {"naive", "reranked", "contextual", "both"}


class RAGPipeline:
    """
    Orchestrates document retrieval, optional re-ranking, and answer
generation.

    Configurations:
        naive:       ChromaDB cosine similarity only; top-5 by score.
        reranked:    ChromaDB top-20 → re-ranker → top-5.
        contextual:  Contextual ChromaDB top-5 by score (no re-ranker).
        both:        Contextual ChromaDB top-20 → re-ranker → top-5.
    """

    def __init__(self, config: str) -> None:
        if config not in VALID_CONFIGS:
            raise ValueError(
                f"Invalid config '{config}'. Must be one of:
{VALID_CONFIGS}."
            )

        self.config = config

        # Select correct ChromaDB collection based on config
```

```
        if config in {"contextual", "both"}:
            collection_name = "contextual_corpus"
        else:
            collection_name = "naive_corpus"

        # Instantiate components
        self.retriever = BaseRetriever(collection_name=collection_name)
        self.reranker = ReRanker() if config in {"reranked", "both"} else
None
        self.generator = Generator()

        logger.info(
            f"RAGPipeline initialised. "
            f"Config='{config}', collection='{collection_name}', "
            f"reranker={'enabled' if self.reranker else 'disabled'}."
        )

    def run(self, query: str) -> dict:
        """
        Execute the full RAG pipeline for a single query.

        Args:
            query: User question string.

        Returns:
            Dict with keys:
                query            (str):        Original query.
                answer           (str):        LLM-generated answer.
                retrieved_chunks (list[dict]): Final top-k chunks passed to
generator.
                config           (str):        Active pipeline
configuration.
        """
        logger.info(f"[{self.config.upper()}] Running pipeline for:
'{query[:80]}'")

        # Stage 1: Retrieve top-20 candidates
        candidates = self.retriever.retrieve(query, top_k=TOP_K_RETRIEVAL)

        # Stage 2: Re-rank or slice to top-5
        if self.reranker is not None:
            top_chunks = self.reranker.rerank(query, candidates,
top_k=TOP_K_FINAL)
        else:
            top_chunks = sorted(candidates, key=lambda x: x.get("score",
0.0), reverse=True)
            top_chunks = top_chunks[:TOP_K_FINAL]

        # Stage 3: Generate answer
        answer = self.generator.generate(query, top_chunks)

        return {
            "query": query,
            "answer": answer,
```

```
            "retrieved_chunks": top_chunks,
            "config": self.config,
        }
```

3.10 production_rag/evaluator.py

```
"""
production_rag/evaluator.py

RAGAS-based evaluation harness for all four RAG pipeline configurations.
Runs faithfulness and answer_relevancy metrics across all eval questions
and saves
a comparison CSV to production_rag/results.csv.

Run as:
    python -m production_rag.evaluator
"""

import os
import pandas as pd
from datasets import Dataset
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy
from sentence_transformers import SentenceTransformer

from config.settings import (
    EVAL_QUESTIONS,
    EVAL_GROUND_TRUTHS,
    CORPUS_PDF_PATH,
    CHROMA_PERSIST_DIR,
    EMBED_MODEL,
    RESULTS_CSV,
)
from utils.logger import setup_logger
from utils.pdf_loader import load_and_chunk_pdf
from utils.chroma_store import get_or_create_collection, upsert_chunks
from production_rag.pipeline import RAGPipeline

logger = setup_logger(__name__)

CONFIGS = ["naive", "reranked", "contextual", "both"]


# ── Indexing helpers
# ──────────────────────────────────────────────────────────────

def index_naive_corpus() -> None:
    """
    Load the PDF, embed all chunks, and upsert them into the 'naive_corpus'
    ChromaDB collection. Safe to call multiple times (upsert is
```

```
idempotent).
    """
    logger.info("Indexing naive corpus...")
    chunks = load_and_chunk_pdf(CORPUS_PDF_PATH)
    collection = get_or_create_collection(
        collection_name="naive_corpus",
        persist_dir=CHROMA_PERSIST_DIR,
    )
    embed_model = SentenceTransformer(EMBED_MODEL)
    embeddings = [
        embed_model.encode(c["text"], convert_to_numpy=True).tolist()
        for c in chunks
    ]
    upsert_chunks(collection, chunks, embeddings)
    logger.info(f"Naive corpus indexed: {len(chunks)} chunks in
'naive_corpus'.")


# ── Eval harness
─────────────────────────────────────────────────────────────


def run_eval_harness() -> pd.DataFrame:
    """
    Run RAGAS evaluation across all 4 configurations and all eval
questions.

    For each (config, question) pair:
      - Runs RAGPipeline(config).run(question)
      - Builds a RAGAS Dataset from the result
      - Evaluates faithfulness and answer_relevancy
      - Appends a result row

    Returns:
        pd.DataFrame with columns:
            question_id, question, config, faithfulness, answer_relevancy,
answer
    """
    rows = []
    total = len(CONFIGS) * len(EVAL_QUESTIONS)
    completed = 0

    for config in CONFIGS:
        logger.info(f"=== Evaluating config: {config.upper()} ===")
        pipeline = RAGPipeline(config=config)

        for q_idx, (question, ground_truth) in enumerate(
            zip(EVAL_QUESTIONS, EVAL_GROUND_TRUTHS), start=1
        ):
            logger.info(
                f"[{config}] Q{q_idx}/{len(EVAL_QUESTIONS)}:
'{question[:60]}'"
            )

            try:
```

```
                result = pipeline.run(question)
                answer = result["answer"]
                context_texts = [c["text"] for c in
result["retrieved_chunks"]]

                # Build RAGAS dataset for this single example
                ragas_data = {
                    "question": [question],
                    "answer": [answer],
                    "contexts": [context_texts],
                    "ground_truth": [ground_truth],
                }
                dataset = Dataset.from_dict(ragas_data)

                scores = evaluate(
                    dataset,
                    metrics=[faithfulness, answer_relevancy],
                )

                faith_score = float(scores["faithfulness"])
                rel_score = float(scores["answer_relevancy"])

            except Exception as exc:
                logger.error(
                    f"Eval failed for config='{config}', Q{q_idx}: "
                    f"{type(exc).__name__}: {exc}. Recording NaN scores."
                )
                faith_score = float("nan")
                rel_score = float("nan")
                answer = "EVAL_ERROR"

            rows.append(
                {
                    "question_id": q_idx,
                    "question": question,
                    "config": config,
                    "faithfulness": faith_score,
                    "answer_relevancy": rel_score,
                    "answer": answer,
                }
            )
            completed += 1
            logger.info(
                f"Progress: {completed}/{total} | "
                f"faithfulness={faith_score:.3f}, answer_relevancy=
{rel_score:.3f}"
            )

    df = pd.DataFrame(rows)
    logger.info("Eval harness complete.")
    return df


# ── Results output
```

```
# ── Save results ──────────────────────────────────────────────────────────────

def save_results(df: pd.DataFrame) -> None:
    """
    Pivot the flat eval dataframe so each row is a question and each column
    is a (config, metric) combination. Saves to RESULTS_CSV.

    Output columns:
        question_id, question,
        naive_faithfulness, naive_relevance,
        reranked_faithfulness, reranked_relevance,
        contextual_faithfulness, contextual_relevance,
        both_faithfulness, both_relevance
    """
    pivot_rows = []

    for q_id in df["question_id"].unique():
        q_df = df[df["question_id"] == q_id]
        question = q_df["question"].iloc[0]
        row: dict = {"question_id": int(q_id), "question": question}

        for config in CONFIGS:
            config_row = q_df[q_df["config"] == config]
            if not config_row.empty:
                row[f"{config}_faithfulness"] = round(
                    float(config_row["faithfulness"].iloc[0]), 4
                )
                row[f"{config}_relevance"] = round(
                    float(config_row["answer_relevancy"].iloc[0]), 4
                )
            else:
                row[f"{config}_faithfulness"] = float("nan")
                row[f"{config}_relevance"] = float("nan")

        pivot_rows.append(row)

    pivot_df = pd.DataFrame(pivot_rows)

    os.makedirs(os.path.dirname(RESULTS_CSV), exist_ok=True)
    pivot_df.to_csv(RESULTS_CSV, index=False)
    logger.info(f"Results saved to: {RESULTS_CSV}")
    logger.info(f"\n{pivot_df.to_string(index=False)}")


# ── Entry point ───────────────────────────────────────────────────────────────

if __name__ == "__main__":
    from dotenv import load_dotenv
    load_dotenv()

    from production_rag.contextualizer import contextualize_corpus

    logger.info("Step 1: Indexing naive corpus.")
```

```
    index_naive_corpus()

    logger.info("Step 2: Indexing contextual corpus (this calls Groq API
for each chunk).")
    contextualize_corpus()

    logger.info("Step 3: Running eval harness across all 4
configurations.")
    results_df = run_eval_harness()

    logger.info("Step 4: Saving pivoted results CSV.")
    save_results(results_df)
```

3.11 tests/test_reranker.py

```
"""
tests/test_reranker.py

Unit tests for the ReRanker class — focuses on the local BM25 + cosine
fallback,
which runs without any GCP credentials.
"""

import pytest
from production_rag.reranker import ReRanker


@pytest.fixture
def sample_chunks():
    return [
        {"chunk_id": "a1", "text": "Annual revenue grew by twelve percent
in fiscal year.", "score": 0.85},
        {"chunk_id": "a2", "text": "The board approved a share repurchase
program worth five hundred million.", "score": 0.80},
        {"chunk_id": "a3", "text": "Operating expenses increased due to
higher headcount in engineering.", "score": 0.75},
        {"chunk_id": "a4", "text": "Customer satisfaction scores reached an
all-time high this quarter.", "score": 0.70},
        {"chunk_id": "a5", "text": "Capital expenditures were focused on
data centre infrastructure.", "score": 0.65},
        {"chunk_id": "a6", "text": "Revenue from international markets
outpaced domestic growth rates.", "score": 0.60},
    ]


def test_local_reranker_returns_top_k(sample_chunks):
    reranker = ReRanker()
    result = reranker._rerank_local("revenue growth", sample_chunks,
top_k=3)
    assert len(result) == 3
```

```
def test_local_reranker_adds_rerank_score(sample_chunks):
    reranker = ReRanker()
    result = reranker._rerank_local("revenue growth", sample_chunks,
top_k=3)
    for chunk in result:
        assert "rerank_score" in chunk
        assert 0.0 <= chunk["rerank_score"] <= 1.0


def test_local_reranker_sorted_descending(sample_chunks):
    reranker = ReRanker()
    result = reranker._rerank_local("revenue growth", sample_chunks,
top_k=4)
    scores = [c["rerank_score"] for c in result]
    assert scores == sorted(scores, reverse=True)


def test_rerank_empty_input():
    reranker = ReRanker()
    result = reranker.rerank("some query", [], top_k=5)
    assert result == []


def test_rerank_fewer_chunks_than_top_k(sample_chunks):
    reranker = ReRanker()
    result = reranker.rerank("revenue", sample_chunks[:2], top_k=5)
    assert len(result) == 2
```

3.12 tests/test_contextualizer.py

```
"""
tests/test_contextualizer.py

Unit tests for the context prefix generator, using a mock Groq client
to avoid real API calls in the test suite.
"""

import pytest
from unittest.mock import MagicMock
from production_rag.contextualizer import generate_context_prefix


def _make_mock_groq(response_text: str):
    """Helper: returns a mock Groq client that yields response_text."""
    mock_client = MagicMock()
    mock_choice = MagicMock()
    mock_choice.message.content = response_text
    mock_response = MagicMock()
```

```
    mock_response.choices = [mock_choice]
    mock_client.chat.completions.create.return_value = mock_response
    return mock_client


def test_generate_context_prefix_returns_string():
    mock_client = _make_mock_groq("This chunk discusses annual revenue
growth in section 2.")
    result = generate_context_prefix("Revenue grew 12%.", "Annual Report
2023", mock_client)
    assert isinstance(result, str)
    assert len(result) > 0


def test_generate_context_prefix_strips_whitespace():
    mock_client = _make_mock_groq("  This chunk is about headcount growth.
\n")
    result = generate_context_prefix("Headcount grew.", "Report",
mock_client)
    assert result == "This chunk is about headcount growth."


def test_generate_context_prefix_fallback_on_api_error():
    mock_client = MagicMock()
    mock_client.chat.completions.create.side_effect = Exception("API
timeout")
    result = generate_context_prefix("Some text.", "Doc", mock_client)
    assert result == ""


def test_groq_called_with_correct_model():
    mock_client = _make_mock_groq("Context sentence.")
    generate_context_prefix("Sample text about revenue.", "Report.pdf",
mock_client)
    call_kwargs = mock_client.chat.completions.create.call_args[1]
    assert call_kwargs["model"] == "llama-3.3-70b-versatile"
    assert call_kwargs["temperature"] == 0
```

3.13 tests/test_evaluator.py

```
"""
tests/test_evaluator.py

Integration-style tests for the evaluator module — uses mocking to avoid
real ChromaDB, PDF, and Groq calls in the CI test suite.
"""

import pytest
import pandas as pd
from unittest.mock import patch, MagicMock
```

```
from production_rag.evaluator import save_results


def _make_flat_df():
    """Minimal flat eval dataframe matching the shape produced by
run_eval_harness."""
    rows = []
    for config in ["naive", "reranked", "contextual", "both"]:
        for q_id in [1, 2]:
            rows.append({
                "question_id": q_id,
                "question": f"Sample question {q_id}",
                "config": config,
                "faithfulness": 0.75,
                "answer_relevancy": 0.80,
                "answer": "Sample answer.",
            })
    return pd.DataFrame(rows)


def test_save_results_creates_csv(tmp_path):
    df = _make_flat_df()
    csv_path = str(tmp_path / "results.csv")

    with patch("production_rag.evaluator.RESULTS_CSV", csv_path):
        save_results(df)

    import os
    assert os.path.exists(csv_path)


def test_save_results_correct_columns(tmp_path):
    df = _make_flat_df()
    csv_path = str(tmp_path / "results.csv")

    with patch("production_rag.evaluator.RESULTS_CSV", csv_path):
        save_results(df)

    result_df = pd.read_csv(csv_path)
    expected_cols = {
        "question_id", "question",
        "naive_faithfulness", "naive_relevance",
        "reranked_faithfulness", "reranked_relevance",
        "contextual_faithfulness", "contextual_relevance",
        "both_faithfulness", "both_relevance",
    }
    assert expected_cols.issubset(set(result_df.columns))


def test_save_results_row_count(tmp_path):
    df = _make_flat_df()
    csv_path = str(tmp_path / "results.csv")

    with patch("production_rag.evaluator.RESULTS_CSV", csv_path):
```

```
        save_results(df)

    result_df = pd.read_csv(csv_path)
    assert len(result_df) == 2  # One row per question_id
```

SECTION 4 — Code Logic & Deep-Dive

1. Chunking Strategy

The `load_and_chunk_pdf` function uses a sliding window over word-level tokens with configurable `chunk_size=500` words and `overlap=50` words. This approach is intentionally simple and robust: it requires no NLP pipeline, works identically across all PDF types (scanned, native, mixed), and produces chunks of predictable token length.

The key constraint is `all-MiniLM-L6-v2`'s maximum input length of **256 word-piece tokens**. At an average English word-to-subword-token ratio of roughly 1.3, a 500-word chunk expands to approximately 650 tokens — well above the model's limit. The model silently truncates inputs exceeding 256 tokens, so the effective embedding only captures the first ~200 words of each chunk. This is a known tradeoff: larger chunks preserve more semantic context for the human reader (and for the LLM generator), but the embedding only summarises the leading portion. The 50-word overlap ensures that sentences near chunk boundaries are represented in at least two chunks, reducing the chance that a key fact falls entirely in the truncated portion of both adjacent chunks.

The alternative — sentence-aware chunking using `spacy` or `nltk.sent_tokenize` — produces more semantically coherent chunk boundaries but adds a dependency and is slower. For an intern project with a 10-page PDF, word-based chunking is the correct tradeoff. For production corpora of hundreds of pages, migrating to `langchain.text_splitter.RecursiveCharacterTextSplitter` with `chunk_size=200` (token-counted) is recommended.

---

2. Two-Collection Architecture

`naive_corpus` and `contextual_corpus` are stored as completely separate ChromaDB collections rather than as a flag in the metadata of a single collection. There are two technical reasons for this design.

First, **index integrity**: ChromaDB's HNSW index is built incrementally. If chunks were tagged with a `type=naive` or `type=contextual` metadata flag in a single collection, every query would still scan the combined index and then filter — an O(n) post-filter that defeats the purpose of approximate nearest-neighbour search. Separate collections give each configuration its own HNSW index, preserving sub-linear query time.

Second, **re-indexing safety**: contextual enrichment is a one-time, expensive operation (one Groq API call per chunk). Keeping the collections separate means the naive baseline is never touched during contextual indexing. If the contextualizer fails partway through, the naive collection is still intact and fully queryable. Restarting contextual indexing only needs to re-process failed chunks, not the entire corpus.

The cost of this architecture is doubled storage and doubled one-time embedding compute. For a 10-page PDF producing ~100 chunks at 384 dimensions per embedding, this is negligible. The tradeoff reverses at corpus sizes above roughly 10 million chunks, where storage cost would justify a more sophisticated design with separate embedding namespaces within a single HNSW index.

---

3. Groq API Payload Construction

Both `contextualizer.py` and `generator.py` use the same Groq chat completions API, but with different system prompts and token budgets.

Contextualizer payload:

```
messages=[
    {"role": "system", "content": "You are a document indexing assistant.
Respond with exactly one sentence (max 25 words)..."},
    {"role": "user", "content": f"Document: {doc_title}\nChunk text:
{chunk_text[:800]}"},
]
temperature=0
max_tokens=60
```

The system prompt is highly constraining: it instructs the model to produce exactly one sentence with a hard word limit. `max_tokens=60` is a hard ceiling that prevents runaway output and keeps latency low. `chunk_text[:800]` truncates the chunk before passing it — this is intentional, because the contextualizer only needs enough text to identify the topic, and sending the full chunk would inflate token usage with no benefit.

Generator payload:

```
messages=[
    {"role": "system", "content": "You are a helpful assistant. Answer
using ONLY the provided context..."},
    {"role": "user", "content": f"Context:\n{context_string}\n\nQuestion:
{query}"},
]
temperature=0
max_tokens=512
```

`temperature=0` in both cases is critical for eval reproducibility. At temperature > 0, the model samples from the probability distribution over next tokens, meaning two identical calls may produce different answers. RAGAS scores the answer against the context and ground truth — if answers vary between runs, RAGAS scores vary too, making it impossible to compare configurations across eval runs. Temperature 0 makes the model deterministic (greedy decoding), so scores are stable across repeated evaluations.

---

4. RAGAS Metrics Explained

Faithfulness measures whether every factual claim in the generated answer is supported by the retrieved context. RAGAS decomposes the answer into individual statements, then for each statement asks the LLM "Is this statement supported by the context?" A faithfulness score of 1.0 means every statement in the answer has explicit support in the retrieved chunks. A score of 0.0 means no statement is supported — the model hallucinated the entire answer. Faithfulness is the primary quality signal for RAG systems because hallucination is the principal failure mode: a model that generates confident-sounding answers unsupported by the retrieved documents is dangerous in any production setting.

Answer relevancy measures whether the answer addresses the question that was asked. RAGAS operationalises this by generating several hypothetical questions from the answer and measuring how similar they are (by cosine distance) to the original question. A score of 1.0 means the answer's hypothetical questions cluster tightly around the original question — it's directly on-topic. A score of 0.0 means the answer discusses something unrelated to the question. It is entirely possible to have a high-faithfulness, low-relevancy answer: the model accurately quotes from the context but the context didn't contain the answer to the question, so the quoted material is off-topic. Conversely, a high-relevancy, low-faithfulness answer means the model answered the question in a relevant way but invented facts not present in the context.

---

5. Re-Ranker Scoring Logic

The local BM25 + cosine hybrid fallback computes a score for each candidate chunk as:

```
hybrid_score = 0.4 × BM25_norm(chunk, query) + 0.6 × cosine_score(chunk,
query)
```

BM25 (Best Match 25) is a probabilistic term-frequency relevance function that rewards chunks containing query terms at higher-than-average frequency while penalising very long chunks. BM25 is particularly good at exact-match recall: if the query contains a rare term like a product name or person name, BM25 reliably surfaces chunks containing that term. `BM25Okapi` from `rank_bm25` implements the standard Okapi BM25 variant with saturation parameter k1=1.5 and length normalization b=0.75.

Why 0.4/0.6? The slight bias toward cosine similarity reflects that for semantic questions ("What was the revenue trend?"), lexical overlap alone is insufficient — the model needs to understand paraphrase. Cosine similarity on sentence-transformer embeddings handles paraphrase well. The 0.4 BM25 weight ensures exact-match recall (entity names, numbers, years) is not completely discarded. If labeled preference data (human relevance judgments) were available, the optimal weights would be found by minimising a ranking loss (e.g., pairwise RankNet or listwise LambdaRank) across the labeled pairs, producing data-driven weights rather than this principled default.

---

6. Graceful Degradation Chain

```
QUERY RECEIVED
     │
     ▼
BaseRetriever.retrieve(query, top_k=20)
```

```
      │
      ├── ChromaDB query fails (disk full, collection empty)
      │         └── Exception bubbles to pipeline.run()
      │             └── RAGPipeline returns error dict
      │
      ▼ [success: 20 candidate chunks returned]
      │
ReRanker.rerank(query, chunks, top_k=5)
      │
      ├── use_vertex == True?
      │         │
      │         ├── Vertex AI API call
      │         │         │
      │         │         ├── GoogleAPICallError / network timeout
      │         │         │         └── [LOG ERROR] fall through to ↓
      │         │         │
      │         │         └── [success] → return Vertex top-5
      │         │
      │         └── Local BM25 + cosine fallback
      │                   └── [always succeeds if chunks is non-empty]
      │                       → return local top-5
      │
      ├── use_vertex == False
      │         └── Local BM25 + cosine fallback directly
      │
      ▼ [top-5 chunks available]
      │
Generator.generate(query, top_5_chunks)
      │
      ├── Groq RateLimitError
      │         └── [LOG ERROR] return "Generation failed due to API error."
      │
      ├── Groq APIError
      │         └── [LOG ERROR] return "Generation failed due to API error."
      │
      ├── Any other Exception
      │         └── [LOG ERROR] return "Generation failed due to API error."
      │
      └── [success] → return answer string

CONTEXTUALIZER (pre-indexing):
      │
generate_context_prefix(chunk, title, groq_client)
      │
      ├── Any Groq exception
      │         └── [LOG ERROR] return ""
      │             chunk indexed as-is (prefix omitted, not dropped)
      │
      └── [success] → prefix prepended, chunk re-embedded, upserted
```

SECTION 5 — Deployment & Execution Guide

Step 1: Prerequisites Check

```
python --version
# Expected: Python 3.10.x or higher
# If lower, install Python 3.10+ from python.org or via pyenv
```

Step 2: Create and Activate `myenv`

macOS / Linux:

```
cd production_rag_project
python -m venv myenv
source myenv/bin/activate
```

Windows:

```
cd production_rag_project
python -m venv myenv
myenv\Scripts\activate
```

You should see `(myenv)` in your shell prompt after activation.

Step 3: Install Dependencies

Create `requirements.txt` with the following content (pinned versions tested together):

```
groq==0.9.0
chromadb==0.5.3
sentence-transformers==3.0.1
ragas==0.1.9
rank-bm25==0.2.2
pdfplumber==0.11.1
pandas==2.2.2
python-dotenv==1.0.1
google-cloud-discoveryengine==0.11.11
datasets==2.20.0
pytest==8.2.2
```

Install:

```
pip install -r requirements.txt
```

Step 4: `.env` File Setup

Create a `.env` file in the project root with the following content:

```
GROQ_API_KEY=your_groq_api_key_here
GOOGLE_CLOUD_PROJECT=your_gcp_project_id
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account.json
VERTEX_LOCATION=global
```

Notes:

- `GROQ_API_KEY` is required. Obtain it at console.groq.com.
- `GOOGLE_CLOUD_PROJECT` and `GOOGLE_APPLICATION_CREDENTIALS` are optional. Leave them blank or omit them entirely to use the local BM25 + cosine fallback re-ranker.
- `VERTEX_LOCATION` defaults to `global` in `settings.py` if not set.

Step 5: Place Corpus PDF

Copy any PDF to the data directory:

```
cp /path/to/your/document.pdf data/corpus.pdf
```

To use a different PDF at runtime, update `CORPUS_PDF_PATH` in `config/settings.py` or pass the path explicitly to `load_and_chunk_pdf()` and `contextualize_corpus()`.

Step 6: Run Contextualizer (One-Time)

This step calls the Groq API once per chunk to generate context prefixes and indexes the enriched chunks into `contextual_corpus`. It only needs to be run once per corpus — the results are persisted in ChromaDB.

```
python -m production_rag.contextualizer
```

Expected output pattern:

```
2024-06-01 09:00:01 | INFO | Starting contextual corpus indexing from:
./data/corpus.pdf
2024-06-01 09:00:02 | INFO | PDF has 10 pages.
```

```
2024-06-01 09:00:02 | INFO | Chunking complete: 87 chunks from 10 pages
(chunk_size=500, overlap=50).
2024-06-01 09:00:04 | INFO | Processed 10/87 chunks.
...
2024-06-01 09:01:45 | INFO | Contextual corpus indexing complete. Total
chunks: 87, Groq API calls: 87, Estimated tokens used: 124,200.
```

Step 7: Run Full Eval Harness

```
python -m production_rag.evaluator
```

This runs all 4 configurations × 8 questions = 32 pipeline executions, each followed by a RAGAS evaluation call. Total runtime depends on Groq API latency (typically 2–5 minutes for the full harness).

Step 8: Verify Outputs

Check that results CSV exists and has correct headers:

```
python -c "import pandas as pd; df =
pd.read_csv('./production_rag/results.csv'); print(df.columns.tolist());
print(df.head())"
```

Expected columns:

```
['question_id', 'question', 'naive_faithfulness', 'naive_relevance',
 'reranked_faithfulness', 'reranked_relevance',
 'contextual_faithfulness', 'contextual_relevance',
 'both_faithfulness', 'both_relevance']
```

Check the runtime log:

```
tail -50 output.log
```

Step 9: Run Tests

```
python -m pytest tests/ -v
```

Expected output:

```
tests/test_reranker.py::test_local_reranker_returns_top_k PASSED
tests/test_reranker.py::test_local_reranker_adds_rerank_score PASSED
tests/test_reranker.py::test_local_reranker_sorted_descending PASSED
tests/test_reranker.py::test_rerank_empty_input PASSED
tests/test_reranker.py::test_rerank_fewer_chunks_than_top_k PASSED
tests/test_contextualizer.py::test_generate_context_prefix_returns_string
PASSED
tests/test_contextualizer.py::test_generate_context_prefix_strips_whitespac
e PASSED
tests/test_contextualizer.py::test_generate_context_prefix_fallback_on_api_
error PASSED
tests/test_contextualizer.py::test_groq_called_with_correct_model PASSED
tests/test_evaluator.py::test_save_results_creates_csv PASSED
tests/test_evaluator.py::test_save_results_correct_columns PASSED
tests/test_evaluator.py::test_save_results_row_count PASSED

12 passed in 3.41s
```

Step 10: Expected Terminal Output During a Full Run

A realistic sample of what `output.log` looks like during a successful eval harness run:

```
2024-06-01 09:05:00 | INFO | Step 1: Indexing naive corpus.
2024-06-01 09:05:00 | INFO | Loading PDF: ./data/corpus.pdf
2024-06-01 09:05:01 | INFO | PDF has 10 pages.
2024-06-01 09:05:01 | INFO | Chunking complete: 87 chunks from 10 pages
(chunk_size=500, overlap=50).
2024-06-01 09:05:01 | INFO | ChromaDB collection 'naive_corpus' ready
(persist_dir='./chroma_store', count=0).
2024-06-01 09:05:03 | INFO | Upserted 87 chunks into collection
'naive_corpus'.
2024-06-01 09:05:03 | INFO | Naive corpus indexed: 87 chunks in
'naive_corpus'.
2024-06-01 09:05:03 | INFO | Step 2: Indexing contextual corpus.
2024-06-01 09:05:03 | INFO | Starting contextual corpus indexing from:
./data/corpus.pdf
2024-06-01 09:05:13 | INFO | Processed 10/87 chunks.
2024-06-01 09:05:23 | INFO | Processed 20/87 chunks.
2024-06-01 09:06:45 | INFO | Contextual corpus indexing complete. Total
chunks: 87, Groq API calls: 87, Estimated tokens used: 124,200.
2024-06-01 09:06:45 | INFO | Step 3: Running eval harness across all 4
configurations.
2024-06-01 09:06:45 | INFO | === Evaluating config: NAIVE ===
2024-06-01 09:06:45 | INFO | RAGPipeline initialised. Config='naive',
collection='naive_corpus', reranker=disabled.
2024-06-01 09:06:45 | INFO | [naive] Q1/8: 'What was the company's total
revenue for the fisc...'
2024-06-01 09:06:45 | INFO | Retrieving top-20 chunks for query: 'What was
the company's total revenue for the fiscal year?'
```

```
2024-06-01 09:06:46 | INFO | Retrieved 20 chunks. Top result: score=0.8421,
preview='Total revenue for fiscal year 2023 reached...'
2024-06-01 09:06:47 | INFO | Answer generated. Tokens: 64. Preview: 'Based
on the provided context, total revenue...'
2024-06-01 09:06:49 | INFO | Progress: 1/32 | faithfulness=0.875,
answer_relevancy=0.912
2024-06-01 09:06:49 | INFO | [naive] Q2/8: 'Which business segment reported
the highest operat...'
...
2024-06-01 09:12:33 | INFO | === Evaluating config: BOTH ===
2024-06-01 09:12:33 | INFO | ReRanker mode: LOCAL fallback (BM25 + cosine
hybrid).
2024-06-01 09:12:33 | INFO | RAGPipeline initialised. Config='both',
collection='contextual_corpus', reranker=enabled.
2024-06-01 09:15:47 | INFO | Progress: 32/32 | faithfulness=0.950,
answer_relevancy=0.938
2024-06-01 09:15:47 | INFO | Eval harness complete.
2024-06-01 09:15:47 | INFO | Step 4: Saving pivoted results CSV.
2024-06-01 09:15:47 | INFO | Results saved to: ./production_rag/results.csv
```

SECTION 6 — Intern Viva & Code Review Questions

```
## Project Evaluation & Code Review — Day 6 Goal 3: Production RAG

### Q1: What is the purpose of `TOP_K_RETRIEVAL = 20` and `TOP_K_FINAL = 5`
in `settings.py`, and why are they different values?
**Answer:**
`TOP_K_RETRIEVAL = 20` is the number of candidate chunks fetched from
ChromaDB by the `BaseRetriever`. This is deliberately large to maximise
recall — we want the truly relevant chunk to be in the candidate set, even
if cosine similarity alone doesn't rank it first. `TOP_K_FINAL = 5` is the
number of chunks actually passed to the Groq generator after re-ranking.
This is small because LLM context windows have a cost (tokens = latency +
API cost) and because passing irrelevant chunks into the prompt degrades
answer quality by introducing noise. The gap between 20 and 5 is the re-
ranker's operating range: it receives 20 candidates and selects the best 5.
If `TOP_K_RETRIEVAL == TOP_K_FINAL`, the re-ranker would have nothing to
filter. In configurations without a re-ranker (`naive`, `contextual`), the
top-20 are sorted by cosine score and sliced to 5 — the same architectural
pattern with a weaker selector.

---

### Q2: In `utils/logger.py`, why do we check `logger.handlers` before
adding new handlers? What bug does this prevent?
**Answer:**
Python's `logging` module caches logger instances by name in a global
registry. If `setup_logger("my_module")` is called twice (which happens
when a module is imported from multiple places, or when a test reloads a
module), `logging.getLogger("my_module")` returns the **same** logger
```

object both times. Without the `if logger.handlers` guard, the function would add a second `StreamHandler` and a second `FileHandler` to the already-configured logger. The result is that every subsequent log message would appear twice in the console and twice in `output.log`. The guard checks whether handlers already exist before adding new ones, so the function is idempotent: subsequent calls return the correctly configured logger unchanged. This is a subtle but real production bug — duplicated log lines are particularly confusing in multi-module projects where the logger names are the same across test runs.

Q3: Explain why the `contextual_corpus` and `naive_corpus` are stored as two separate ChromaDB collections rather than using a metadata flag on the same collection.
Answer:
There are two reasons. First, **query efficiency**: ChromaDB's HNSW index supports approximate nearest-neighbour search in sub-linear time, but only against all vectors in a collection. If both naive and contextual chunks were in a single collection tagged with a `type` metadata field, filtering by `type=contextual` would require a full scan followed by metadata filtering — an O(n) operation that defeats the purpose of HNSW. Separate collections give each configuration its own HNSW graph, preserving logarithmic query time. Second, **indexing safety**: contextualisation is expensive (one Groq API call per chunk, one re-embedding). Keeping it in a separate collection means a failed or partial contextualisation run never corrupts the naive baseline. The naive pipeline remains fully operational at any point, even while contextual indexing is in progress or has failed partway through.

Q4: In `generator.py`, why is `temperature=0` used during evaluation? What would happen to your RAGAS scores if you used `temperature=0.9`?
Answer:
`temperature=0` forces greedy decoding — the model always picks the highest-probability next token. This makes generation deterministic: the same input always produces the same output. During evaluation, reproducibility is essential: if you re-run the eval harness to check a result, you need confidence that score differences between runs reflect pipeline changes, not stochastic variation in the LLM output. At `temperature=0.9`, the model samples from the top-probability distribution, introducing randomness. Two runs with identical inputs could produce meaningfully different answers. RAGAS `faithfulness` and `answer_relevancy` scores would then vary between runs — sometimes by 5–10 percentage points — making it impossible to determine whether a `contextual` config is genuinely better than `naive` or whether you simply got a lucky sample. For production serving, `temperature=0.7` is typical for diverse responses, but for any benchmarking or A/B comparison, `temperature=0` is the correct choice.

Q5: Walk through exactly what happens in `reranker.py` when

`GOOGLE_APPLICATION_CREDENTIALS` is not set. What code path executes and what scoring formula is used?
Answer:
At module load time, `_CREDENTIALS_PATH = os.environ.get("GOOGLE_APPLICATION_CREDENTIALS", "")` returns an empty string, so `_USE_VERTEX = bool("" and VERTEX_PROJECT_ID)` evaluates to `False`. When `ReRanker.__init__` runs, `self.use_vertex = False`, and the logger records that the local fallback is active. When `rerank()` is called, the `if self.use_vertex:` branch is skipped entirely; `_rerank_local()` is called directly. Inside `_rerank_local`, the query and each chunk's text are tokenized by `.lower().split()`. A `BM25Okapi` model is fit on the tokenized corpus. `bm25.get_scores(query_tokens)` returns a raw BM25 score for each chunk. These are normalized to `[0,1]` by dividing by `max(bm25_scores)`. The final hybrid score is computed as `0.4 * bm25_norm[i] + 0.6 * cosine_score` where `cosine_score` is the original ChromaDB cosine similarity score stored in `chunk["score"]`. Chunks are sorted descending by `hybrid_score`, the top `top_k` are returned with a new key `rerank_score`, and the function logs the input/output counts and the top score.

Q6: RAGAS `faithfulness` and `answer_relevancy` measure different things. Give a concrete example of a RAG response that scores high on relevancy but low on faithfulness.
Answer:
Query: "What was the company's net income for fiscal year 2023?"

Retrieved context: "Operating expenses increased by 15% in FY2023 due to expanded headcount and infrastructure investments. Revenue grew to $4.2 billion."

Generated answer: "The company's net income for fiscal year 2023 was $840 million, reflecting a 20% profit margin on $4.2 billion in revenue."

***Relevancy score: high (~0.9)** — The answer directly addresses the question about net income and fiscal year 2023. RAGAS would generate hypothetical questions from this answer like "What was the net income in FY2023?" and "What was the profit margin?" — both very similar to the original query.*

***Faithfulness score: low (~0.1)** — The context mentions revenue ($4.2B) and rising operating expenses, but never states net income or a profit margin. The model fabricated the $840M figure and the 20% margin. Neither statement has support in the retrieved context. This is the canonical RAG hallucination failure mode: the model answers confidently and relevantly by extrapolating beyond the evidence.*

Q7: The `contextualizer.py` generates a 1-line prefix per chunk using the Groq API. What is the total number of Groq API calls made during a one-time contextual indexing run for a 100-chunk corpus, and what is the ongoing cost per query compared to the naive pipeline?

Answer:
During the one-time contextual indexing run for a 100-chunk corpus, exactly **100 Groq API calls** are made — one call to `generate_context_prefix()` per chunk. This is a one-time cost that is amortised across all future queries against the contextual corpus. The contextual collection is indexed once and persisted; subsequent queries use the pre-indexed enriched embeddings with no additional Groq calls during retrieval.

At query time, the comparison is: **naive pipeline** makes exactly 1 Groq API call (the generator call in `generator.py`). The **contextual pipeline** also makes exactly 1 Groq API call at query time — the same generator call. The context prefix generation is a pre-indexing step, not a per-query step. Therefore, the ongoing per-query cost in Groq API calls is identical for naive and contextual configurations. The contextual pipeline's cost advantage (compared to an approach that generates prefixes at query time) is precisely why pre-indexing was chosen: the one-time cost is fully amortised, and the retrieval quality improvement is free at query time.

Q8: If your `results.csv` shows that `contextual` alone outperforms `both` (contextual + reranked) on `faithfulness`, what are three plausible technical explanations for this regression?
Answer:
First, **re-ranker score miscalibration**: the BM25 + cosine hybrid re-ranker (active when Vertex AI credentials are absent) uses fixed weights of 0.4/0.6 that may not be optimal for a contextually-enriched corpus. The context prefix changes the lexical and semantic distribution of each chunk's text. BM25 scores against the original query may systematically down-rank enriched chunks that now contain prefix language ("This chunk discusses...") that doesn't match query tokens, leading the re-ranker to select fewer relevant chunks than the naive top-5 by cosine score.

Second, **context dilution**: the re-ranker filters from 20 candidates to 5. If the contextual embeddings are already very well-calibrated (the whole point of contextual retrieval), the top-5 by cosine similarity may already be optimal — and re-ranking introduces noise by sometimes swapping in a lower-cosine chunk that happens to score well on BM25. The `both` configuration's re-ranker is fighting against an already-good retrieval, introducing degradation at the margin.

Third, **RAGAS faithfulness sensitivity to context set composition**: faithfulness is scored as the fraction of answer statements supported by the retrieved context. If the `both` pipeline's top-5 includes even one off-topic chunk (a re-ranking error), the generator may use language from that chunk that doesn't directly support the answer to the question. The `contextual` pipeline's cosine-selected top-5 may happen to form a more coherent, focused context set for these specific eval questions, yielding higher faithfulness for this particular corpus and question set.

Q9: The BM25 + cosine hybrid fallback re-ranker uses a 0.4/0.6

weighting. Describe a principled experiment you would run to determine the optimal weights for a specific production corpus, without using labeled preference data.
Answer:
*This is a hyperparameter tuning problem over the weight scalar `α` in `hybrid_score = α * BM25_norm + (1−α) * cosine_score`. Without labeled preference data, we can use an unsupervised proxy metric: **normalized discounted cumulative gain (NDCG) against the ground-truth answers in the eval set**.*

The experiment: for `α` in `{0.0, 0.1, 0.2, ..., 1.0}`, run the full reranked eval configuration with the current `α`. For each (query, top-5) pair, score the top-5 chunks by measuring which chunk's text contains the most n-gram overlap with the corresponding ground truth answer (using ROUGE-1 or BM25 score of the ground truth against each chunk — no human labels needed). Compute NDCG@5 for each `α` using these proxy relevance scores. Select the `α` that maximises average NDCG@5 across all eval questions. This is valid because the proxy (ground truth n-gram overlap with chunk text) is strongly correlated with actual retrieval quality, and it requires only the existing eval questions and ground truth answers — no human annotation of individual chunks. Run the experiment three times with different random seeds for the BM25 tokenization to confirm the result is stable, then report the optimal `α` with its confidence interval.

Q10: You are presenting `lift_report.md` to a client. The `both` configuration improves faithfulness by 18% but adds 100 extra API calls per query (20 Vertex re-rank calls + 1 contextual prefix per new doc). Design a decision framework — with specific cost and latency thresholds — for recommending whether a client should ship `naive`, `reranked`, `contextual`, or `both`, given their use case and scale.
Answer:
*The decision framework operates on four axes: **latency budget**, **query volume**, **faithfulness requirement**, and **operational complexity tolerance**.*

Ship `naive` if:
- Latency budget is under 500ms end-to-end and the corpus is static (no re-indexing needed)
- Faithfulness requirement is below 0.70 (informational use cases where occasional imprecision is acceptable)
- Query volume exceeds 10,000/day and cost per query is the primary constraint (no re-ranking API cost)
- The team has no GCP infrastructure

Ship `reranked` if:
- Faithfulness requirement is 0.70–0.85
- Latency budget is 500ms–1,500ms (Vertex AI re-ranking adds ~200–600ms)
- GCP credentials and a project with Discovery Engine API enabled are available
- Query volume is under 100,000/day (Vertex ranking API pricing applies per request)
- The corpus is not enriched (no pre-indexing budget for contextualisation)

Ship `contextual` if:
- Faithfulness requirement is 0.80–0.90
- The corpus is largely static (contextualisation is a one-time cost)
- Latency budget is 500ms or less at query time (contextual adds no per-query latency)
- GCP infrastructure is not available (no Vertex AI access)
- Documents are long, multi-section reports where chunk-level context loss is a known problem

Ship `both` if:
- Faithfulness requirement exceeds 0.90 (legal, medical, financial compliance use cases where hallucination is a liability)
- Query volume is under 50,000/day (to contain Vertex ranking API cost)
- Latency budget is 1,500–3,000ms (contextual re-indexing is one-time; Vertex re-ranking adds ~400ms at query time)
- The client has a GCP project and engineering resources to maintain the Vertex AI integration
- The 18% faithfulness improvement over naive justifies the infrastructure investment (frame this as: if one hallucination costs the client $X in liability or customer churn, and the system serves Y queries/day, the expected cost reduction must exceed the Vertex API spend)

The recommendation in the lift report should include: observed faithfulness delta across all four configs, per-query Vertex API cost estimate at the client's projected query volume, and a break-even analysis showing the minimum faithfulness improvement required to justify the cost of moving from each tier to the next.